

Claude Code 开源代码深度解读

基于 Anthropic 2025 年开源的 Claude Code 源码（`../claude-code/src/`），1902 个 TypeScript 文件。本文档作为课程后续讲解的事实依据，所有结论附带文件路径和行号。

1. 核心 Agent Loop: `query.ts`

1.1 主循环结构

文件: `src/query.ts`，函数 `queryLoop()`，行 241-1729

循环体是 `while (true)`（行 307），不是 `while (stop_reason === 'tool_use')`。

循环状态定义（行 204-217）：

```
type State = {
  messages: Message[]
  toolUseContext: ToolUseContext
  autoCompactTracking: AutoCompactTrackingState | undefined
  maxOutputTokensRecoveryCount: number
  hasAttemptedReactiveCompact: boolean
  maxOutputTokensOverride: number | undefined
  pendingToolUseSummary: Promise<ToolUseSummaryMessage | null> | undefined
  stopHookActive: boolean | undefined
  turnCount: number
  transition: Continue | undefined // 上一轮为什么 continue
}
```

1.2 `stop_reason` 不被信任

行 556-558，有一条关键注释：

```
// Note: stop_reason === 'tool_use' is unreliable -- it's not always set
correctly.
const toolUseBlocks: ToolUseBlock[] = []
let needsFollowUp = false
```

CC 用 `needsFollowUp` 布尔值替代 `stop_reason` 检查。当 streaming 过程中检测到 `tool_use` block 时设为 true（行 829-835）：

```
const msgToolUseBlocks = message.message.content.filter(
  content => content.type === 'tool_use',
) as ToolUseBlock[]
if (msgToolUseBlocks.length > 0) {
  toolUseBlocks.push(...msgToolUseBlocks)
  needsFollowUp = true
}
```

课程启示：我们讲 stop_reason 驱动循环是正确的教学简化，但需补充工程实践中的鲁棒做法。

1.3 循环退出/继续的完整地图

12 个退出点 (return)：

行号	原因	触发条件
648	blocking_limit	token 数超过阻断限制
977	image_error	图片大小/resize 错误
996	model_error	不可恢复的 API/模型错误
1051	aborted_streaming	用户在 streaming 中中断
1175	prompt_too_long	413 错误，context collapse 无法恢复
1182	prompt_too_long	context collapse 后仍然太长
1264	completed	API 错误消息（无 stop hooks）
1279	stop_hook_prevented	stop hook 阻止继续
1357	completed	正常 end_turn，stop hooks 通过
1515	aborted_tools	用户在工具执行中中断
1521	hook_stopped	hook 阻止继续
1711	max_turns	达到 maxTurns 限制

7 个继续点 (continue)：

行号	原因	触发条件
1115	collapse_drain_retry	context collapse drain 后重试

行号	原因	触发条件
1165	<code>reactive_compact_retry</code>	reactive compact 成功后重试
1221	<code>max_output_tokens_escalate</code>	升级到 64k max output tokens
1251	<code>max_output_tokens_recovery</code>	注入 “resume” 消息重试
1305	<code>stop_hook_blocking</code>	stop hook 返回阻断错误
1341	<code>token_budget_continuation</code>	token budget 未用完，继续
1727	<code>next_turn</code>	主循环继续：工具结果收集完毕，进入下一轮

1.4 工具结果的构造与回传

工具执行在 `src/services/tools/toolOrchestration.ts` (行 19-82)：- 只读工具并发执行（最多 10 个）- 写工具串行执行

每个工具结果作为 `UserMessage` (含 `tool_result` block) 返回，在 `query.ts` 行 1384-1408 收集：

```
for await (const update of toolUpdates) {
  if (update.message) {
    yield update.message
    toolResults.push(
      ...normalizeMessagesForAPI(
        [update.message],
        toolUseContext.options.tools,
      ).filter(_ => _.type === 'user'),
    )
  }
}
```

在行 1716 合并为下一轮的输入：

```
messages: [...messagesForQuery, ...assistantMessages, ...toolResults],
```

1.5 调用者如何消费循环

`queryLoop()` 是 async generator，调用者用 `for await` 消费：

- **REPL 交互模式:** `src/screens/REPL.tsx` 行 2793-2803
- **SDK/Headless:** `src/QueryEngine.ts` 行 675-686

2. 工具集

2.1 核心工具（始终加载）

工具	Wire Name	文件	只读	延迟加载
Agent	Agent	tools/AgentTool/AgentTool.tsx	No	No
Bash	Bash	tools/BashTool/BashTool.tsx	依赖命令	No
Glob	Glob	tools/GlobTool/GlobTool.ts	Yes	No
Grep	Grep	tools/GrepTool/GrepTool.ts	Yes	No
Read	Read	tools/FileReadTool/ FileReadTool.ts	Yes	No
Edit	Edit	tools/FileEditTool/ FileEditTool.ts	No	No
Write	Write	tools/FileWriteTool/ FileWriteTool.ts	No	No
Skill	Skill	tools/SkillTool/SkillTool.ts	-	No
Brief	SendUserMessage	tools/BriefTool/BriefTool.ts	Yes	No

2.2 延迟加载工具（通过 ToolSearch 按需发现）

src/tools/ToolSearchTool/ToolSearchTool.ts

25+ 个工具标记 `shouldDefer: true`，不随初始 system prompt 发送 schema。模型用 ToolSearch 发现后才加载。

包括：TaskOutput, ExitPlanMode, NotebookEdit, WebFetch, TodoWrite, WebSearch, TaskStop, AskUserQuestion, EnterPlanMode, SendMessage 等。

课程启示：这是 tool schema token 成本问题的工程解法。我们在 2.3 节讨论了 “每次 API 调用发送 5K-10K tokens 的工具定义” 的 trade-off，CC 的 ToolSearch 是折中方案——核心工具始终加载（~9 个），其余按需加载。

2.3 条件工具（feature flag 控制）

工具	条件
TaskCreate/Get/Update/List	<code>isTodoV2Enabled()</code>
LSP	<code>ENABLE_LSP_TOOL</code> 环境变量
EnterWorktree/ExitWorktree	<code>isWorktreeModeEnabled()</code>
TeamCreate/TeamDelete	<code>isAgentSwarmsEnabled()</code>
PowerShell	<code>isPowerShellToolEnabled()</code> (Windows)
Sleep, Cron*, Monitor 等	内部 feature flag

完整工具池约 50+ 个。

3. Edit 工具：search/replace 实现细节

文件: `src/tools/FileEditTool/`

3.1 参数 Schema（`types.ts`）

```
z.strictObject({
  file_path: z.string(),
  old_string: z.string(),
  new_string: z.string(),
  replace_all: z.boolean().default(false).optional(),
})
```

确认使用 `old_string/new_string` search/replace 范式。

3.2 匹配流程（`utils.ts`）

1. 精确匹配优先： `fileContent.includes(searchString)`
2. 引号规范化 fallback（`findActualString()`）：将 curly quotes 和 straight quotes 统一后匹配

- 3. 唯一性检查: `old_string` 匹配多处且 `replace_all=false` 时, 拒绝执行
- 4. XML 反消毒 (`normalizeFileEditInput()`): 处理模型输出的被消毒 XML 标签
- 5. staleness 检查: 编辑前校验文件自上次 Read 后是否被修改

3.3 引号风格保留 (`preserveQuoteStyle()`)

当通过引号规范化找到匹配时, `new_string` 也会被转换为文件原有的引号风格。

课程启示: 我们的 MVP Edit 实现了核心的 search/replace, 但缺少引号规范化和 staleness 检查。前者对小模型尤其重要 (7B 模型常混淆引号类型)。

4. Permission 模型

文件: `src/utils/permissions/permissions.ts` (~1487 行), `src/types/permissions.ts` (~442 行)

4.1 权限模式

模式	行为
<code>default</code>	危险操作需确认
<code>acceptEdits</code>	文件编辑自动允许, shell 命令需确认
<code>bypassPermissions</code>	跳过所有检查
<code>dontAsk</code>	需确认的操作自动拒绝
<code>plan</code>	计划模式, 需审批后执行
<code>auto</code>	AI 分类器判断是否安全
<code>bubble</code>	将权限请求冒泡给父 agent

4.2 规则来源 (8 个)

`userSettings`, `projectSettings`, `localSettings`, `flagSettings`, `policySettings`, `cliArg`, `command`, `session`

4.3 决策流水线（`hasPermissionsToUseToolInner()`，行 1158）

Step 1: Deny/Ask 规则检查

1a. 整个工具是否被 deny rule 禁止

1b. 整个工具是否有 ask rule

1c. 工具自身的 `checkPermissions()` 返回值

1d-1g. 内容级规则、安全路径检查（`.git/`，`.claude/` 等）

Step 2: 模式决策

2a. `bypassPermissions` 模式直接允许

2b. 检查 `always-allow` 规则

Step 3: 默认 `passthrough` → 提示用户

4.4 自动允许的工具（不需用户确认）

Glob, Grep, Read, WebSearch, LSP, TodoWrite, TaskCreate/Get/Update/List, EnterPlanMode, ExitPlanMode, Brief

4.5 需要用户确认的工具

Bash（非只读命令）, Edit, Write, NotebookEdit, Agent（特定模式）, MCP 工具, WebFetch（非预批准域名）

课程启示：CodeAct 一节中我们说“harness 看到 Read 可以允许、看到 Bash 可以拦截——工具名本身就是权限标签”，CC 源码完美证实了这个论点。实际实现比我们描述的更精细（支持内容级规则如 `Bash(git *)`），可在第三讲展开。

5. System Prompt 组装

文件: `src/constants/prompts.ts`（~915 行）

5.1 组装顺序

静态段（可缓存，放在 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之前）：

顺序	函数	内容
1	<code>getSimpleIntroSection()</code> 行 175	身份 + 安全指令

顺序	函数	内容
2	<code>getSimpleSystemSection()</code> 行 186	输出渲染、权限模式、hooks
3	<code>getSimpleDoingTasksSection()</code> 行 199	编码行为规则（不 gold-plate、先读后改）
4	<code>getActionsSection()</code> 行 255	可逆性/爆炸半径，危险操作列表
5	<code>getUsingYourToolsSection()</code> 行 269	工具使用指令（用 Read 不用 cat）
6	<code>getSimpleToneAndStyleSection()</code> 行 430	不用 emoji、简洁
7	<code>getOutputEfficiencySection()</code> 行 403	“Go straight to the point”

动态段（每次重算）：

顺序	内容
8	Agent tool 指导、AskUserQuestion、skill commands
9	Auto-memory 系统提示
10	环境信息（CWD、git status、platform、model）
11	MCP 服务器指令
12	语言/输出风格

5.2 优先级系统（`systemPrompt.ts` 行 41）

```
Override > Coordinator > Agent > Custom > Default
appendSystemPrompt 始终追加在末尾
```

5.3 缓存分界

`SYSTEM_PROMPT_DYNAMIC_BOUNDARY`（行 114）将 prompt 分为全局可缓存段和会话特定段。Anthropic API 的 prompt cache 只对 boundary 之前的段生效。

课程启示：我们的 system prompt 设计可以参考 CC 的分段策略——静态指令放前面（利用缓存），动态信息放后面。

6. Subagent 架构

文件: `src/tools/AgentTool/`

6.1 三条生成路径

路径	触发条件	隔离方式	文件
Normal	指定 <code>subagent_type</code>	新 API 调用，独立上下文	<code>runAgent.ts</code>
Fork	未指定 <code>type</code> + <code>fork</code> 启用	继承父对话，共享 <code>prompt cache</code>	<code>forkSubagent.ts</code>
Teammate	<code>swarm</code> 模式	<code>tmux/iTerm2</code> 进程 或 <code>in-process AsyncLocalStorage</code>	<code>AgentTool.tsx</code> + <code>spawnInProcess.ts</code>

6.2 上下文隔离

`createSubagentContext()` (`src/utils/forkedAgent.ts`) : - 克隆 `readFileState` - 创建子 `AbortController` - 独立的工具状态集合 - 可选共享: `shareSetAppState`, `shareAbortController`

6.3 通信机制

- **SendMessage 工具** (`src/tools/SendMessageTool/SendMessageTool.ts`) : 文件级 mailbox
- **任务通知**: 完成时生成 `<task-notification>` XML 消息
- **广播**: `to: "*"` 发送给所有活跃 teammate

6.4 任务存储

`src/utils/tasks.ts` : - JSON 文件存储在 `~/.claude/tasks/<taskListId>/<taskId>.json` - `proper-lockfile` 文件锁防止并发冲突 - 团队内所有 teammate 共享同一个 task list

7. Context 管理

7.1 六层压缩策略

层级	机制	文件
1	Tool result budget	<code>query.ts</code> <code>applyToolResultBudget()</code>
2	Snip compact	feature-gated 历史裁剪
3	Microcompact	清除旧的 tool result 内容
4	Context collapse	投射压缩视图
5	Auto-compact	完整对话摘要（阈值 = contextWindow - 33K）
6	Reactive compact	API 返回 prompt-too-long 时紧急压缩

7.2 Auto-compact 阈值

`src/services/compact/autoCompact.ts` 行 72:

```
autocompactThreshold = effectiveContextWindowSize - 13,000
effectiveContextWindowSize = contextWindow - 20,000
```

熔断器：连续失败 3 次后停止重试（`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES`）。

7.3 Compact 摘要格式

`src/services/compact/prompt.ts`：摘要包含 9 个结构化段落：1. Primary Request and Intent 2. Key Technical Concepts 3. Files and Code Sections 4. Errors and fixes 5. Problem Solving 6. All user messages 7. Pending Tasks 8. Current Work 9. Optional Next Step

课程启示：这正是我们当前对话使用的 compact 格式——“continuation from a previous conversation” 的 Summary 就是这个模板生成的。

8. Nudge / 恢复机制

8.1 Max Output Tokens 恢复 (query.ts 行 1188-1256)

当模型撞到 max_output_tokens: 1. 第一次: 升级到 ESCALATED_MAX_TOKENS = 64K 2. 然后最多 3 次恢复, 每次注入:

```
"Output token limit hit. Resume directly -- no apology, no recap of what
you were doing.
Pick up mid-thought if that is where the cut happened. Break remaining
work into smaller pieces."
```

8.2 Stop Hooks (query/stopHooks.ts)

用户配置的 shell 命令, 在模型 end_turn 时执行: - 返回 blocking error → 注入消息, 强制模型继续 - 返回 non-blocking feedback → 信息性输出 - 阻止继续 → 终止循环

8.3 Token Budget 续写 (query/tokenBudget.ts)

用户指定 token budget (如 +500k) 时自动续写: - 已用 < 90% 且未出现 diminishing returns → 注入 nudge 消息继续 - Diminishing returns 检测: 连续 3 次 delta < 500 tokens → 停止

9. CLAUDE.md / Memory 加载

文件: src/utils/claudemd.ts

9.1 加载顺序 (行 1-26)

1. **Managed:** /etc/claude-code/CLAUDE.md (全局策略)
2. **User:** ~/.claude/CLAUDE.md + ~/.claude/rules/*.md (用户全局)
3. **Project:** CLAUDE.md, .claude/CLAUDE.md, .claude/rules/*.md (项目级, checked in)
4. **Local:** CLAUDE.local.md (项目级, 不 checked in)
5. **AutoMem:** MEMORY.md (自动记忆系统)

9.2 目录遍历

`getMemoryFiles()` (行 790) : 从 CWD 向上遍历到根目录, 离 CWD 越近优先级越高。

9.3 @include 指令

支持 `@path`, `@./relative`, `@~/home`, `@/absolute`。最大递归深度 5。

10. 模型路由与分级

文件: `/usr/lib/node_modules/@anthropic-ai/claude-code/cli.js` (v2.1.92, minified/bundled)

10.1 模型目录 (Di 对象)

CC 内部定义了完整的模型目录, 每个模型有多 provider ID 映射:

内部别名	firstParty ID	定价 (per M tokens, input/output)
<code>opus46</code>	<code>claude-opus-4-6</code>	\$5 / \$25 (普通) , \$30 / \$150 (fast mode)
<code>sonnet46</code>	<code>claude-sonnet-4-6</code>	\$3 / \$15
<code>haiku45</code>	<code>claude-haiku-4-5-20251001</code>	\$1 / \$5

还包括旧版模型 (`opus40/41/45`, `sonnet35/37/40/45`, `haiku35`) , 以及 `bedrock/vertex/foundry` 等第三方 provider 的 ID 映射。

10.2 默认模型选择 (DN() 函数)

```
DN() {
  if (KC()) // Claude Max 订阅
    return fN() + (yJ() ? "[1m]" : "") // Opus 4.6 + optional 1M context
  if (Y16()) // Team plan with max-5x tier
    return fN() + (yJ() ? "[1m]" : "") // Opus 4.6
  return nT() // Sonnet 4.6 (所有其他用户)
}
```

只有 Max 和 Team-5x 用户默认 Opus, 其余全部默认 Sonnet 4.6。

三层 Override 优先级（qc() 函数）：`runtime override > ANTHROPIC_MODEL env > settings.json model`

10.3 别名解析（y5() 函数）

别名	解析为
"opus"	fN() → Opus 4.6
"sonnet"	nT() → Sonnet 4.6
"haiku"	l06() → Haiku 4.5
"best"	fN() → Opus 4.6
"opusplan"	nT() → Sonnet 4.6（注意：非 plan 模式时解析为 Sonnet）

支持 [1m] 后缀启用 1M context window（如 "opus[1m]"）。

10.4 Sub-Agent 模型选择（JE6() 函数）

```
JE6(requestedModel, mainLoopModel, override, permissionMode) {
  if (CLAUDE_CODE_SUBAGENT_MODEL env) → 强制使用
  if (override exists && same family as main) → 使用主模型
  if (requestedModel == null) → "inherit" → 继承父模型 (via hu())
  if (requestedModel same family as main) → 使用主模型
  else → y5(requestedModel) 解析别名
}
```

默认 "inherit"：子 Agent 继承父 Agent 的主循环模型。Ym4() 函数检查请求的别名是否与主模型同 family，避免重复解析。

10.5 Plan 模式模型调整（hu() 函数）

```
hu({ permissionMode, mainLoopModel, exceeds200kTokens }) {
  if (override == "opusplan" && mode == "plan" && !exceeds200k)
    return Opus 4.6 // plan 模式用 Opus
  if (override == "haiku" && mode == "plan")
    return Sonnet 4.6 // haiku 在 plan 模式自动升级
  return mainLoopModel // 其他情况保持原模型
}
```

opusplan 的设计：plan 阶段用 Opus 做深度推理（高认知负荷，少 token），执行阶段自动降级为 Sonnet（低认知负荷，多 token）。

10.6 Haiku 专属用途——“Small Fast Model”（EJ() 函数）

```
EJ() = process.env.ANTHROPIC_SMALL_FAST_MODEL || Haiku 4.5
```

Haiku 不参与主对话循环，专门用于轻量任务：hook prompt 回调、WebFetch 内容摘要、web search 结果处理、轻量 verification agents。

10.7 Fast Mode（内部代号“penguin”）

- **条件:** Opus 4.6 + firstParty API + 未被 org 禁用
- **效果:** 添加 fast-mode-2026-02-01 beta header
- **定价:** 6 倍普通 Opus 4.6 价格（\$30/\$150 vs \$5/\$25）
- **限制:** 429 rate limit 时自动 cooldown 回退到普通模式
- **本质:** 同一个 Opus 4.6 模型，6 倍价格换更快输出速度，不是换模型

NJ() 函数检查模型是否为 opus-4-6 ——Fast mode 仅 Opus 4.6 可用。

10.8 Effort Level

仅 Opus 4.6 和 Sonnet 4.6 支持。“max” effort 仅 Opus 4.6 可用。有效值：“low” / “medium” / “high” / “max”。

10.9 关键环境变量

变量	效果
ANTHROPIC_MODEL	覆盖默认模型
ANTHROPIC_DEFAULT_OPUS_MODEL	覆盖 "opus" 别名解析目标
ANTHROPIC_DEFAULT_SONNET_MODEL	覆盖 "sonnet" 别名解析目标
ANTHROPIC_DEFAULT_HAIKU_MODEL	覆盖 "haiku" 别名解析目标
ANTHROPIC_SMALL_FAST_MODEL	覆盖轻量任务模型（默认 Haiku）
CLAUDE_CODE_SUBAGENT_MODEL	强制所有子 Agent 使用指定模型
CLAUDE_CODE_DISABLE_FAST_MODE	禁用 Fast mode

变量	效果
CLAUDE_CODE_EFFORT_LEVEL	设置 effort level

课程启示：CC 的模型路由是“编排需要强模型，执行可以用弱模型”的工业级实证。
opusplan 模式、sub-agent model 参数、Haiku 专属轻量任务——三层金字塔设计揭示了一个原则：模型选择不是全局统一的决策，而是按任务角色分级的分发策略。

11. 与课程/MVP 的对照总结

课程论点验证

课程论点	CC 源码验证	结论
stop_reason 驱动循环	CC 不信任 stop_reason，用 needsFollowUp	补充工程实践
7 个核心工具	核心 9 个始终加载 + 25+ 延迟加载	补充 ToolSearch
Edit search/replace	完全确认 + 引号规范化 + staleness 检查	增强描述
harness 驱动循环	12 个退出点 + 7 个继续点	增强论据
工具名即权限标签	完全确认，支持内容级规则 Bash(git *)	增强论据
Adapter 层	CC 不需要（直连 Claude API），MVP 特有	无需调整
Subagent = 新 API 调用	完全确认 + Fork/Teammate 两种优化路径	可选扩展
编排用强模型，执行用弱模型	opusplan + sub-agent model 参数 + Haiku 轻量任务	三层金字塔实证

MVP 改进建议

改进	优先级	对应 CC 代码
检查 tool_use block 而非仅 stop_reason	高	query.ts 行 829-835
Edit staleness 检查	中	FileEditTool.ts
	中	autoCompact.ts

改进	优先级	对应 CC 代码
Context 压缩（至少 auto-compact）		
引号规范化	低	<code>FileEditTool/Utils.ts</code> <code>findActualString()</code>
ToolSearch 延迟加载	低	<code>ToolSearchTool.ts</code>